

React Interview Questions & Answers

Table of Contents

1. [React Fundamentals](#)
 2. [Components & JSX](#)
 3. [State & Props](#)
 4. [Hooks](#)
 5. [Event Handling](#)
 6. [Performance Optimization](#)
 7. [Advanced Patterns](#)
 8. [Testing](#)
-

React Fundamentals

1. What is React and what are its key features?

Answer: React is a JavaScript library for building user interfaces, particularly web applications. It was created by Facebook and is now maintained by Meta and the open-source community.

Key Features:

- **Component-Based Architecture:** Build encapsulated components that manage their own state
- **Virtual DOM:** Efficient updates through a virtual representation of the DOM
- **Declarative:** Describe what the UI should look like for any given state
- **Unidirectional Data Flow:** Data flows down, events flow up
- **JSX:** JavaScript syntax extension for writing HTML-like code
- **Ecosystem:** Rich ecosystem with tools like React Router, Redux, etc.

jsx

```
// Simple React component
function Welcome({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Usage
function App() {
  return (
    <div>
      <Welcome name="Alice" />
      <Welcome name="Bob" />
    </div>
  );
}
```

2. What is the Virtual DOM and how does it work?

Answer: The Virtual DOM is a JavaScript representation of the actual DOM (Document Object Model). It's a programming concept where a "virtual" representation of the UI is kept in memory and synced with the "real" DOM.

How it works:

1. **Virtual Representation:** React creates a virtual representation of the DOM tree
2. **Diffing:** When state changes, React creates a new virtual DOM tree and compares it with the previous tree
3. **Reconciliation:** React calculates the minimum set of changes needed
4. **Update:** Only the necessary changes are applied to the actual DOM

jsx

// Before state change

```
const virtualDOM1 = {  
  type: 'div',  
  props: {  
    children: [  
      { type: 'h1', props: { children: 'Count: 0' } },  
      { type: 'button', props: { children: 'Increment' } }  
    ]  
  }  
};
```

// After state change

```
const virtualDOM2 = {  
  type: 'div',  
  props: {  
    children: [  
      { type: 'h1', props: { children: 'Count: 1' } }, // Only this changed  
      { type: 'button', props: { children: 'Increment' } }  
    ]  
  }  
};
```

// React will only update the h1 element's text content

3. What is JSX and how does it work?

Answer: JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code within JavaScript. It's transformed into regular JavaScript function calls.

jsx

// JSX

```
const element = <h1 className="greeting">Hello, world! </h1>;
```

// Transforms to (using React 17+)

```
const element = React.createElement(  
  'h1',  
  { className: 'greeting' },  
  'Hello, world!'  
);
```

// More complex example

```
const Welcome = ({ name, age }) => (  
  <div className="user-card">  
    <h2>Welcome, {name}! </h2>  
    {age >= 18 && <p>You are an adult </p>}  
    <ul>  
      {['React', 'JavaScript', 'CSS'].map(skill => (  
        <li key={skill}>{skill} </li>  
      ))}  
    </ul>  
  </div>  
);
```

// JSX Rules:

// 1. Must return a single parent element or Fragment

```
const Valid = () => (  
  <>  
    <h1>Title </h1>  
    <p>Content </p>  
  </>  
);
```

// 2. Use className instead of class

// 3. Use camelCase for attributes

// 4. Self-closing tags must end with />

```
const SelfClosing = () => <input type="text" placeholder="Name" />;
```

// 5. Embed JavaScript expressions with {}

```
const Dynamic = ({ items }) => (  
  <div>  
    <p>Total items: {items.length} </p>  
    {items.length > 0 ? (  
      <ul>  
        {items.map(item => <li key={item.id}>{item.name} </li>)}  
      )  
    }  
  </div>  
);
```

```
    </ul>
  ):(
    <p>No items found</p>
  )}
</div>
);
```

Components & JSX

4. What's the difference between Functional and Class Components?

Answer:

Class Components (Legacy):

jsx

```
class Welcome extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    this.setState({ count: this.state.count + 1 });
  }

  componentDidMount() {
    console.log('Component mounted');
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
      console.log('Count updated');
    }
  }

  render() {
    return (
      <div>
        <h1>Count: {this.state.count}</h1>
        <button onClick={this.handleClick}>Increment</button>
      </div>
    );
  }
}
```

Functional Components (Modern):

jsx

```
import { useState, useEffect } from 'react';

function Welcome() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  };

  useEffect(() => {
    console.log('Component mounted');
  }, []);

  useEffect(() => {
    if (count > 0) {
      console.log('Count updated');
    }
  }, [count]);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}

// Even more concise with modern patterns
const WelcomeModern = () => {
  const [count, setCount] = useState(0);

  useEffect(() => console.log('Component mounted'), []);
  useEffect(() => count > 0 && console.log('Count updated'), [count]);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
};
```

Key Differences:

- **Syntax:** Functional components are simpler and more concise
- **State:** Class components use `this.state`, functional use `useState`
- **Lifecycle:** Class components use lifecycle methods, functional use `useEffect`
- **Performance:** Functional components with hooks are generally more performant
- **Modern Standard:** Functional components are the current best practice

5. Explain React Component Lifecycle

Answer:

Class Component Lifecycle:

jsx

```
class LifecycleExample extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0, data: null };
    console.log('1. Constructor - Component initialized');
  }

  static getDerivedStateFromProps(props, state) {
    console.log('2. getDerivedStateFromProps - Sync state with props');
    // Return object to update state, or null for no update
    return null;
  }

  componentDidMount() {
    console.log('3. componentDidMount - Component mounted');
    // Perfect for:
    // - API calls
    // - Setting up subscriptions
    // - DOM manipulation
    this.fetchData();
  }

  shouldComponentUpdate(nextProps, nextState) {
    console.log('4. shouldComponentUpdate - Should component re-render?');
    // Return false to prevent re-render
    return nextState.count !== this.state.count;
  }

  getSnapshotBeforeUpdate(prevProps, prevState) {
    console.log('5. getSnapshotBeforeUpdate - Capture info before DOM update');
    // Return value passed to componentDidUpdate
    return prevState.count;
  }

  componentDidUpdate(prevProps, prevState, snapshot) {
    console.log('6. componentDidUpdate - Component updated');
    console.log('Previous count:', snapshot);

    // Perfect for:
    // - API calls based on prop changes
    // - DOM manipulation after update
    if (prevState.count !== this.state.count) {
      this.logCountChange();
    }
  }
}
```

```
}  
  
componentWillUnmount() {  
  console.log('7. componentWillUnmount - Component will be destroyed');  
  // Perfect for:  
  // - Cleanup timers  
  // - Cancel API requests  
  // - Remove event listeners  
  this.cleanup();  
}
```

```
static getDerivedStateFromError(error) {  
  console.log('Error boundary - getDerivedStateFromError');  
  return { hasError: true };  
}
```

```
componentDidCatch(error, errorInfo) {  
  console.log('Error boundary - componentDidCatch');  
  // Log error to service  
}
```

```
fetchData = async () => {  
  try {  
    const data = await fetch('/api/data');  
    this.setState({ data });  
  } catch (error) {  
    console.error('Error fetching data:', error);  
  }  
};
```

```
logCountChange = () => {  
  console.log('Count changed to: ${this.state.count}');  
};
```

```
cleanup = () => {  
  // Cleanup operations  
};
```

```
render() {  
  console.log('Render - Component rendering');  
  return (  
    <div>  
      <h1>Count: {this.state.count}</h1>  
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

```
};  
}  
}
```

Functional Component with Hooks Equivalent:

jsx

```
import { useState, useEffect, useRef } from 'react';

function LifecycleHooksExample() {
  const [count, setCount] = useState(0);
  const [data, setData] = useState(null);
  const prevCountRef = useRef();

  // Constructor equivalent
  console.log('Component function called (like constructor)');

  // componentDidMount equivalent
  useEffect(() => {
    console.log('Component mounted');
    fetchData();

    // componentWillUnmount equivalent (cleanup)
    return () => {
      console.log('Component will unmount - cleanup');
      cleanup();
    };
  }, []); // Empty dependency array = run once on mount

  // componentDidUpdate equivalent
  useEffect(() => {
    console.log('Component updated - count changed');
    if (prevCountRef.current !== undefined) {
      logCountChange();
    }
    prevCountRef.current = count;
  }, [count]); // Run when count changes

  // Combined mount and update
  useEffect(() => {
    console.log('This runs on mount and every render');
  }); // No dependency array = run on every render

  const fetchData = async () => {
    try {
      const response = await fetch('/api/data');
      const result = await response.json();
      setData(result);
    } catch (error) {
      console.error('Error fetching data:', error);
    }
  }
}
```

```

};

const logCountChange = () => {
  console.log(`Count changed from ${prevCountRef.current} to ${count}`);
};

const cleanup = () => {
  // Cleanup operations
};

console.log('Render - Component rendering');

return (
  <div>
    <h1>Count: {count}</h1>
    <button onClick={() => setCount(c => c + 1)}>
      Increment
    </button>
    {data && <p>Data loaded: {JSON.stringify(data)}</p>}
  </div>
);
}

```

State & Props

6. What's the difference between State and Props?

Answer:

Props (Properties):

- Data passed from parent to child components
- Immutable (read-only) within the component
- Used to configure and customize components
- Flow down the component tree (unidirectional data flow)

State:

- Internal data managed by the component
- Mutable (can be changed)
- Triggers re-renders when updated
- Local to the component (unless lifted up)

jsx

// Parent component

```
function App() {  
  const [theme, setTheme] = useState('light');  
  const [user, setUser] = useState({ name: 'John', role: 'admin' });  
  
  return (  
    <div>  
      /* Props: theme and user are passed down */  
      <Header theme={theme} user={user} />  
      <MainContent theme={theme} onThemeChange={setTheme} />  
    </div>  
  );  
}
```

// Child component receiving props

```
function Header({ theme, user }) {  
  // Props are read-only  
  // theme = 'dark'; // ❌ This would cause an error  
  
  return (  
    <header className={`header ${theme}`}>  
      <h1>Welcome, {user.name}! </h1>  
      <span>Role: {user.role}</span>  
    </header>  
  );  
}
```

// Child component with its own state

```
function MainContent({ theme, onThemeChange }) {  
  // State: local to this component  
  const [isMenuOpen, setIsMenuOpen] = useState(false);  
  const [articles, setArticles] = useState([]);  
  
  // Component can modify its own state  
  const toggleMenu = () => {  
    setIsMenuOpen(!isMenuOpen);  
  };  
  
  // Component can call prop functions to communicate up  
  const handleThemeToggle = () => {  
    const newTheme = theme === 'light' ? 'dark' : 'light';  
    onThemeChange(newTheme); // Calling parent's function  
  };  
}
```

```

useEffect(() => {
  // Fetch articles on mount
  fetchArticles().then(setArticles);
}, []);

return (
  <main className={`main ${theme}`}>
    <button onClick={toggleMenu}>
      {isMenuOpen ? 'Close Menu' : 'Open Menu'}
    </button>

    <button onClick={handleThemeToggle}>
      Switch to {theme === 'light' ? 'Dark' : 'Light'} Theme
    </button>

    {isMenuOpen && (
      <nav>
        <ul>
          {articles.map(article => (
            <li key={article.id}>{article.title}</li>
          ))}
        </ul>
      </nav>
    )}
  </main>
);
}

```

```

// Props validation with PropTypes
import PropTypes from 'prop-types';

```

```

Header.propTypes = {
  theme: PropTypes.oneOf(['light', 'dark']).isRequired,
  user: PropTypes.shape({
    name: PropTypes.string.isRequired,
    role: PropTypes.string.isRequired
  }).isRequired
};

```

```

// Default props
Header.defaultProps = {
  theme: 'light'
};

```

Key Differences Summary:

Aspect	Props	State
Mutability	Immutable	Mutable
Origin	Passed from parent	Created in component
Purpose	Configure component	Manage component data
Triggers Re-render	When parent updates	When setState is called
Scope	Available to child	Local to component

7. How do you handle state updates correctly?

Answer:

Basic State Updates:

jsx

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  // ✅ Correct: Direct value  
  const increment = () => {  
    setCount(count + 1);  
  };  
  
  // ✅ Better: Using functional update (prevents stale closure issues)  
  const incrementBetter = () => {  
    setCount(prevCount => prevCount + 1);  
  };  
  
  // ❌ Wrong: Multiple updates in same render  
  const incrementTwiceWrong = () => {  
    setCount(count + 1); // Will be batched  
    setCount(count + 1); // Still uses old count value  
    // Result: count increases by 1, not 2  
  };  
  
  // ✅ Correct: Multiple updates with functional updates  
  const incrementTwiceCorrect = () => {  
    setCount(prevCount => prevCount + 1);  
    setCount(prevCount => prevCount + 1);  
    // Result: count increases by 2  
  };  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={increment}>Increment</button>  
      <button onClick={incrementBetter}>Increment Better</button>  
      <button onClick={incrementTwiceCorrect}>Increment Twice</button>  
    </div>  
  );  
}
```

Object State Updates:

jsx

```
function UserProfile() {
  const [user, setUser] = useState({
    name: 'John',
    email: 'john@example.com',
    address: {
      street: '123 Main St',
      city: 'New York'
    },
    preferences: {
      theme: 'light',
      notifications: true
    }
  });

  // ❌ Wrong: Direct mutation
  const updateNameWrong = (newName) => {
    user.name = newName; // Mutating state directly
    setUser(user); // Won't trigger re-render
  };

  // ✅ Correct: Shallow copy for top-level properties
  const updateName = (newName) => {
    setUser({
      ...user,
      name: newName
    });
  };

  // ✅ Correct: Deep update for nested objects
  const updateCity = (newCity) => {
    setUser({
      ...user,
      address: {
        ...user.address,
        city: newCity
      }
    });
  };

  // ✅ Correct: Using functional update for complex logic
  const updatePreferences = (key, value) => {
    setUser(prevUser => ({
      ...prevUser,
      preferences: {
        [key]: value
      }
    }));
  };
}
```

```

        ...prevUser.preferences,
        [key]: value
    }
    }));
};

//  Alternative: Using useReducer for complex state
const [userState, dispatch] = useReducer(userReducer, initialUser);

return (
  <div>
    <h1>{user.name}</h1>
    <p>{user.email}</p>
    <p>{user.address.street}, {user.address.city}</p>

    <button onClick={() => updateName('Jane')}>
      Change Name
    </button>
    <button onClick={() => updateCity('Los Angeles')}>
      Change City
    </button>
    <button onClick={() => updatePreferences('theme', 'dark')}>
      Toggle Theme
    </button>
  </div>
);
}

```

// useReducer example for complex state management

```

function userReducer(state, action) {
  switch (action.type) {
    case 'UPDATE_NAME':
      return { ...state, name: action.payload };
    case 'UPDATE_EMAIL':
      return { ...state, email: action.payload };
    case 'UPDATE_ADDRESS':
      return {
        ...state,
        address: { ...state.address, ...action.payload }
      };
    case 'UPDATE_PREFERENCE':
      return {
        ...state,
        preferences: {
          ...state.preferences,
          [action.key]: action.value
        }
      };
  }
}

```

```
};  
default:  
  return state;  
}  
}
```

Array State Updates:

jsx

```
function TodoList() {  
  const [todos, setTodos] = useState([]);  
  const [inputValue, setInputValue] = useState("");  
  
  // ✅ Adding items  
  const addTodo = () => {  
    if (inputValue.trim()) {  
      setTodos(prevTodos => [  
        ...prevTodos,  
        {  
          id: Date.now(),  
          text: inputValue,  
          completed: false  
        }  
      ]);  
      setInputValue("");  
    }  
  };  
  
  // ✅ Removing items  
  const removeTodo = (id) => {  
    setTodos(prevTodos => prevTodos.filter(todo => todo.id !== id));  
  };  
  
  // ✅ Updating items  
  const toggleTodo = (id) => {  
    setTodos(prevTodos =>  
      prevTodos.map(todo =>  
        todo.id === id  
          ? { ...todo, completed: !todo.completed }  
          : todo  
      )  
    );  
  };  
  
  // ✅ Updating item at specific index  
  const updateTodoText = (index, newText) => {  
    setTodos(prevTodos =>  
      prevTodos.map((todo, i) =>  
        i === index ? { ...todo, text: newText } : todo  
      )  
    );  
  };  
  
  // ✅ Creating a new todo
```

```
//  Sorting/reordering
const sortTodos = () => {
  setTodos(prevTodos => [...prevTodos].sort((a, b) => a.text.localeCompare(b.text)));
};

return (
  <div>
    <input
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      onKeyPress={(e) => e.key === 'Enter' && addTodo()}
    />
    <button onClick={addTodo}>Add Todo</button>
    <button onClick={sortTodos}>Sort Todos</button>

    <ul>
      {todos.map((todo, index) => (
        <li key={todo.id}>
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => toggleTodo(todo.id)}
          />
          <span style={{
            textDecoration: todo.completed ? 'line-through' : 'none'
          }}>
            {todo.text}
          </span>
          <button onClick={() => removeTodo(todo.id)}>Remove</button>
        </li>
      ))}
    </ul>
  </div>
);
}
```

Hooks

8. Explain useState and how it works

Answer:

`useState` is a Hook that allows you to add state to functional components. It returns an array with two elements: the current state value and a function to update it.

jsx

```
import { useState } from 'react';

function StateExamples() {
  // Basic usage
  const [count, setCount] = useState(0);

  // With object state
  const [user, setUser] = useState({
    name: '',
    email: '',
    age: 0
  });

  // With array state
  const [items, setItems] = useState([]);

  // Lazy initial state (for expensive computations)
  const [expensiveValue, setExpensiveValue] = useState(() => {
    console.log('This only runs once on mount');
    return someExpensiveComputation();
  });

  // Multiple state variables
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [data, setData] = useState(null);

  // Event handlers
  const handleIncrement = () => {
    setCount(count + 1);
  };

  const handleIncrementByAmount = (amount) => {
    setCount(prevCount => prevCount + amount);
  };

  const handleUserUpdate = (field, value) => {
    setUser(prevUser => ({
      ...prevUser,
      [field]: value
    }));
  };

  const handleAddItem = (item) => {
    // ...
  }
}
```

```

setItems(prevItems => [...prevItems, item]);
};

// Async state updates
const handleAsyncOperation = async () => {
  setLoading(true);
  setError(null);

  try {
    const result = await fetchData();
    setData(result);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

return (
  <div>
    <h2>Count: {count}</h2>
    <button onClick={handleIncrement}>Increment</button>
    <button onClick={() => handleIncrementByAmount(5)}>+5</button>
    <button onClick={() => setCount(0)}>Reset</button>

    <div>
      <input
        placeholder="Name"
        value={user.name}
        onChange={(e) => handleUserUpdate('name', e.target.value)}
      />
      <input
        placeholder="Email"
        value={user.email}
        onChange={(e) => handleUserUpdate('email', e.target.value)}
      />
      <input
        type="number"
        placeholder="Age"
        value={user.age}
        onChange={(e) => handleUserUpdate('age', parseInt(e.target.value))}
      />
    </div>

    <button onClick={() => handleAddItem(`Item ${items.length + 1}`)}>
      Add Item
    </button>
  </div>
);

```

```

    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>

    <button onClick={handleAsyncOperation} disabled={loading}>
      {loading ? 'Loading...' : 'Fetch Data'}
    </button>
    {error && <p style={{color: 'red'}}>Error: {error}</p>}
    {data && <p>Data: {JSON.stringify(data)}</p>}
  </div>
);
}

```

// Custom hook using useState

```

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  const reset = () => setCount(initialValue);
  const setValue = (value) => setCount(value);

  return {
    count,
    increment,
    decrement,
    reset,
    setValue
  };
}

```

// Usage of custom hook

```

function CounterWithCustomHook() {
  const { count, increment, decrement, reset } = useCounter(10);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

```

```
function someExpensiveComputation() {  
  let result = 0;  
  for (let i = 0; i < 1000000; i++) {  
    result += i;  
  }  
  return result;  
}  
  
async function fetchData() {  
  const response = await fetch('/api/data');  
  return response.json();  
}
```

9. Explain useEffect and its use cases

Answer:

`useEffect` is a Hook that lets you perform side effects in functional components. It serves the same purpose as `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` combined.

jsx

```
import { useState, useEffect, useRef } from 'react';

function EffectExamples() {
  const [count, setCount] = useState(0);
  const [user, setUser] = useState(null);
  const [windowWidth, setWindowWidth] = useState(window.innerWidth);
  const intervalRef = useRef();

  // 1. Effect without dependencies - runs after every render
  useEffect(() => {
    console.log('This runs after every render');
    document.title = `Count: ${count}`;
  });

  // 2. Effect with empty dependencies - runs once on mount
  useEffect(() => {
    console.log('Component mounted');
    fetchUserData();

    // Cleanup function - runs on unmount
    return () => {
      console.log('Component will unmount');
    };
  }, []); // Empty dependency array

  // 3. Effect with dependencies - runs when dependencies change
  useEffect(() => {
    console.log('Count changed to:', count);

    if (count > 0) {
      localStorage.setItem('count
```